

CMDSolver

Modular Nonlinear Equation System Solver
for Aerothermodynamic Systems

Technical Documentation

Package: apps.eqnParser · 46 Java Source Files · April 2026

GraalVM 25 / OpenJDK 21 · CASprzak 0.3.0 · GSPAR (Grund/WIAS 1994)

GSPAR Sparse LU Solver (Grund, WIAS Berlin ~1994) — Java Port
Newton-Armijo · Levenberg-Marquardt · Broyden · Homotopy
Symbolic Jacobian Assembly via CASprzak CAS Library
Validated on 623-equation Aerothermodynamic Benchmark System

Table of Contents

1.	Introduction	3
2.	Quick Start	3
3.	System Architecture	4
3.1	Parser Layer	4
3.2	Solver Strategy Pattern	5
3.3	GSPAR Sparse Solver Architecture	5
3.4	Thermodynamic Built-in Functions	6
4.	Newton-Armijo Solver	6
5.	Levenberg-Marquardt Solver	7
6.	Broyden Quasi-Newton Solver	7
7.	Homotopy / Arc-Length Continuation	8
8.	GSPAR Sparse LU Solver	8
9.	Command-Line Reference	10
10.	CAS File Format	11
11.	Development History	12
12.	Bug Fix Log	13
13.	Open Issues	15
14.	Lessons Learned	17
15.	Roadmap	19

1. Introduction

CMDSolver is a modular, command-line-driven solver for large nonlinear equation systems with a particular focus on aerothermodynamic process simulation. It follows the equation-oriented approach pioneered by simulators such as SpeedUp (later Aspen Technology): rather than solving component models sequentially, *all equations are solved simultaneously* as a single nonlinear system.

The solver accepts equation systems described in a structured text format (.cas files), builds symbolic Jacobian matrices via the CASprzak computer algebra library, and provides multiple Newton-type solution strategies suitable for systems ranging from a few dozen to many hundreds of equations.

Design Goals

- Equation-oriented simulation of complex aerothermodynamic systems (gas turbines, jet engines, mixers, nozzles)
- Robustness under poor initial conditions and near-singular Jacobians ($\kappa \sim 10^{14}$)
- Pluggable solver strategy pattern — solvers are interchangeable at runtime via -S: flag
- Transparent logging and diagnostic output (solver.log)
- Extensibility towards block-structured large sparse systems ($n > 10,000$)

Dependency	Version	Purpose
CASprzak	0.3.0	Symbolic differentiation for Jacobian construction
GraalVM / JDK	25 (user), 21 JRE (CI)	Java runtime
GSPAR	Grund/WIAS Berlin ~1994	Sparse LU factorisation (ported from Fortran)

2. Quick Start

Basic Solve

```
java CASSolver -I:engine.cas
```

Solver Selection

```
# Newton with Armijo line search (default) java CASSolver -I:engine.cas
-S:NEWTON_ARMIJO # Levenberg-Marquardt java CASSolver -I:engine.cas
-S:LEVENBERG_MARQUARDT # GSPAR sparse LU (recommended for n > 200) java CASSolver
-I:engine.cas -S:NEWTON_SPARSE # Homotopy for critical Ma=1 transitions java
CASSolver -I:nozzle.cas -S:HOMOTOPY -SH:CRITICAL
```

Output

Results are written to solver.log in the working directory and to stdout. Each iteration logs the current residual norm fsum and iteration counter ITER. The log level is INFO by default.

3. System Architecture

```
cmd/ CASSolver.java <- Entry point, CLI argument parsing apps/eqnParser/
EqnSysReader.java <- .cas file parser (lexer + state machine) EqnData.java <-
Single equation / init-data record EqnSystem.java <- Jacobian assembly, derivative
matrix SystemData.java <- Parsed system: variables + equations
InitGuessApplier.java <- Applies INIT block start values SolverLogger.java <-
Centralised logging facade SolverException.java <- Exception hierarchy
buildInFunctions/ Bifunctions.java <- ~80 thermodynamic built-in functions
ThermoConstants.java <- JANAF/ISA physical constants matrix/ MatrixSimple.java <-
Dense n x n matrix + Gauss-Jordan solver DerivativeMatrix.java <- Symbolic
derivative table solver/ AbstractSolver.java <- Common infrastructure
NewtonArmijoSolver.java <- Newton + Armijo line search LevenbergMarquardtSolver <-
LM with adaptive damping BroydenSolver.java <- Rank-1 Broyden quasi-Newton
HomotopySolver.java <- Arc-length continuation GSPARSolver.java <- Newton + GSPAR
sparse LU gspar/ SparseMatrix.java <- CSR format + buildFromTriplets
GSPARWorkspace.java <- Internal LU arrays GSPAR.java <- Core: factorize, solve,
estimateCondition GSPARPseudoCode.java <- Pseudo-code generation + interpretation
```

3.1 Parser Layer

The parser implements a line-by-line state machine in EqnSysReader. It reads .cas files and populates an EqnSysReaderDataBlock which is then finalised into a SystemData object. After parsing, unused variables are identified and removed — keeping the Jacobian as sparse as possible and forming the foundation for future hierarchical dot-notation variable names.

Symbolic derivatives are computed once via CASprzak's FunctionParser and stored in a DerivativeMatrix. The Jacobian is evaluated numerically each iteration by substituting current variable values.

3.2 Solver Strategy Pattern

All solvers extend AbstractSolver which provides evaluateF(), buildJacobian(), applyStep(), PhysicalProjector (bound clamping), and ResidualNormalizer. Concrete solvers override only computeStep() which returns the Newton step vector dx.

Component	Class	Responsibility
Strategy Interface	SolverStrategy	solve(SystemData, boolean) : List
Configuration	SolverConfig	Immutable builder-pattern: tolerance, maxIter, pivot strategy
Factory	SolverFactory	SolverType enum → SolverStrategy instance
Infrastructure	AbstractSolver	evaluateF, buildJacobian, applyStep, PhysicalProjector

3.3 GSPAR Sparse Solver Architecture

Phase	When	Cost	Methods
Structure analysis	Once (first iteration)	$O(\text{nnz} \cdot \text{fill-in})$	dllugc → dllugb → dlluga → dllugd → dldsp1
Numerical solve	Every Newton iteration	$O(\text{nnz})$	dlfatr → dldint (pseudo-code) → dlsint

4. Newton-Armijo Solver

Flag: -S:NEWTON_ARMIJ0 (default)

Classical Newton-Raphson with Armijo backtracking line search. Each iteration: (1) Build full Jacobian $J(x_k)$, (2) Solve $J \cdot dx = -F$ via Gauss-Jordan, (3) Armijo search for step size α , (4) Update $x_{k+1} = x_k + \alpha \cdot dx$.

Verified: Converges on the 623-equation aerothermodynamic benchmark in ~12 iterations — 2 more than the pre-refactoring version. The extra iterations are the overhead of the Armijo line search, which provides global convergence guarantees absent in the original damped Newton.

5. Levenberg-Marquardt Solver

Flag: -S:LEVENBERG_MARQUARDT

Adaptive damping strategy. Solves the modified system $(J^T J + \mu I) \cdot dx = -J^T \cdot F$ where μ is adapted via the gain ratio $\rho = \text{actual reduction} / \text{predicted reduction}$. Recommended for systems near a sonic transition ($Ma \rightarrow 1$) where the Jacobian becomes singular. μ is reduced by 1/3 on good steps and multiplied by 4 on rejected steps.

6. Broyden Quasi-Newton Solver

Flag: -S:BROYDEN

Rank-1 Broyden update avoids full Jacobian rebuilds between resets. The update: $J_{k+1} = J_k + (y_k - J_k \cdot s_k) \cdot s_k^T / (s_k^T \cdot s_k)$, where $s_k = x_{k+1} - x_k$ and $y_k = F(x_{k+1}) - F(x_k)$.

■ **Current Limitation:** Broyden diverges on the 623-equation benchmark with multi-step updates because $\kappa(J) \approx 10^{14}$ causes the rank-1 updates to corrupt the descent direction after one step. With `broydenResetInterval=1` (the current default), Broyden reduces to Newton-Armijo and converges correctly. Root fix requires GMRES as the inner linear solver — see Open Issues §13.1.

7. Homotopy / Arc-Length Continuation

Flag: -S:HOMOTOPY

Embeds the original system $F(x) = 0$ into a parameter family $H(x,t) = (1-t) \cdot F(x_0) + t \cdot F(x) = 0$ and traces the solution path from $t=0$ to $t=1$ using arc-length parameterisation to navigate past turning points. Each step: predictor (tangent direction) + corrector (inner Newton solve). Ideal for systems with critical parameter transitions.

8. GSPAR Sparse LU Solver

Flag: -S:NEWTON_SPARSE

Newton's method using the GSPAR sparse LU factoriser (Prof. Friedrich Grund, WIAS Berlin, ~1994). For $n=623$ equations with ~10 non-zeros per row, the Jacobian density is only ~1.6%, making sparse methods approximately 50× faster than dense Gauss-Jordan.

Pivot Strategies

MPIV	Name	Description
1 (default)	MIN_FILL	Minimize fill-in, $O(N \cdot M)$ cost analysis
2	MARKOWITZ	Markowitz cost minimisation
3	COLUMN	Column pivoting
4	NONE	No pivoting (fastest, least robust)

Fortran Bug Fixes Applied During Porting

#	Routine	Original Bug	Fix	Source
1	DLPCY	$j+(icr(j)+1)*2+1$	+2 instead of +1	Borchardt 01.07.1998
2	DLLUGE	$NWOR \geq 10 \cdot N + 4 \cdot MA$	$11 \cdot N + 4 \cdot MA$	Borchardt 10.06.1996
3	DLLUGB	$KOSTEN = N \cdot N$	Integer.MAX_VALUE	Borchardt 10.06.1996
4	DLLUGE IPAR(9)	$10 \cdot N + 4 \cdot MLU$	$11 \cdot N + 4 \cdot MLU$	Borchardt 10.06.1996
5	DLSOLA	X(I)=B(I) after DLSOLK overwrote backward substitution	Removed	Independent
6	DLNORI	Missing $JE \neq 0$ guard before ILU[JE]	Added while-loop guard	Independent

9. Command-Line Reference

Input / Output (alle Solver)

Flag	Beschreibung
-I:	Pflicht. *.cas/*.dat = Single-Run; *.csv = Batch-Mode (automatisch)
-U:	Variable-Update-Datei — überschreibt Werte/Bounds aus der CAS-Datei
-C:	CAS-Basismodell für Batch-Mode
-DV:	Startvektordump im -U Format
-h	Hilfe ausgeben und beenden

Solver-Auswahl (-S:)

Flag	Solver	Skalierung
-S:NEWTON_ARMIGO	Newton-Raphson + Armijo (Standard)	NONE
-S:BROYDEN	Broyden Quasi-Newton adaptiv	NONE
-S:LEVENBERG_MARQUARDT	Levenberg-Marquardt	DIAGONAL
-S:HOMOTOPY	Homotopie / Arc-Length	DIAGONAL (LM)
-S:NEWTON_SPARSE	Newton + GSPAR Sparse LU	NONE

Allgemeine Optionen (alle Solver)

Flag	Beschreibung
-SN	Relative Residuumsnormierung $\ F\ /\ F(x_0)\ $
-SC:NONE	Keine Skalierung (Default Newton/Broyden/Sparse)
-SC:DIAGONAL	Diagonale Jacobi-Skalierung (Default LM/Homotopy)
-DI	Startwert-Diagnose + Optimierung: Residuumsdiagnose, uninit. Variablen, Propagation, Sensitivität

Broyden-Optionen (-S:BROYDEN)

Flag	Default	Beschreibung
-SB:RESET=n	1	Jacobi-Reset-Intervall. 10 = adaptiver Broyden-Modus

Flag	Default	Beschreibung
-SB:KAPPA=x	1e12	Konditionszahl-Schwelle für erzwungenen Rebuild

Homotopie-Optionen (-S:HOMOTOPY)

Flag	Default	Beschreibung
-SH:STEPS=n	10	Anzahl Fortsetzungsschritte t=0 bis 1
-SH:DS=x	0.1	Initiale Bogenlängen-Schrittweite
-SH:INNER=typ	LM	Innerer Solver: NEWTON_ORMIJO, LEVENBERG_MARQUARDT, BROYDEN
-SH:CRITICAL	—	Preset für Ma=1 Übergänge: 50 Schritte, ds=0.02

Batch-Mode Optionen (-I:*.csv)

Flag	Beschreibung
-BA	Alle Variablen ausgeben (auch Fixed). Default: nur freie.
-C:	CAS-Basismodell (alternativ: gleichnamige *.cas neben *.csv)

Kombinations-Übersicht

Solver	Kompatible Zusatz-Flags
NEWTON_ORMIJO	-SN, -SC:*, -DI, -U, -DV
BROYDEN	-SB:RESET, -SB:KAPPA, -SN, -SC:*, -DI, -U, -DV
LEVENBERG_MARQUARDT	-SN, -SC:*, -DI, -U, -DV
HOMOTOPY	-SH:STEPS, -SH:DS, -SH:INNER, -SH:CRITICAL, -SN, -SC:*, -DI, -U
NEWTON_SPARSE	-SN, -SC:*, -DI, -U, -DV
Batch (*.csv)	-C, -BA, -S:*, -SN, -SC:*

10. CAS File Format

The .cas format is a structured text language parsed by EqnSysReader. Variables are declared with full metadata; equations are written in implicit form $F(x)=0$ using standard mathematical notation with built-in function calls.

```
VARIABLE T_in AS REAL(Value:288.15; Unit:[K]; Lower:200; Upper:2000;) p_in AS
REAL(Value:101325; Unit:[Pa]; Lower:1; Upper:1e7;) Ma AS REAL(Value:0.5; Unit:[-];
Lower:0; Upper:10;) T_st AS REAL(Value:270; Unit:[K]; Status:Fixed) END EQUATION
T_st = T_in / (1 + (GAMMA-1)/2 * Ma^2); END INIT T_in: Value=290; Lower=200;
```

```
Upper=2000; END
```

Block Keyword	Purpose
VARIABLE ... END	Variable declarations with name, value, bounds, unit, status
EQUATION ... END	Equations in implicit form (left – right = 0)
INIT ... END	Initial guess overrides — applied before solve via InitGuessApplier
PROCEDURE ... END	Built-in thermodynamic function call block
FOR ... END	Loop expansion for array-indexed variables
IF ... ELSE ... END	Conditional equations (active based on current values)

11. Development History

Phase 1 — Original Monolithic Solver

The original codebase consisted of a single `EqnSystem.java` with all parsing, Jacobian assembly, and Newton iteration inlined. The solver used a damped Newton method without line search and was tightly coupled to the Swing GUI.

- GUI-coupled: solver code imported Swing classes (`HTMLDocument`, `JTextPane`)
- Fixed-point damping (0.5–1.0) instead of adaptive line search
- All variables tracked by name string — aliases not separated from names
- No exception hierarchy — errors returned as status flags
- Broyden variant (`NewtonBroydenGMRES`) existed as a separate class using GMRES

Phase 2 — Structural Refactoring

Concerns separated into distinct classes:

- `InitGuessApplier` extracted from `EqnSystem.run()`
- `EqnSysReaderGui` separated from backend `EqnSysReader` (no Swing imports in backend)
- `SolverLogger` introduced as a centralised `java.util.logging` facade
- `SolverException` hierarchy: `ParseException`, `SystemDimensionException`, `SolverRuntimeException`, `SingularJacobianException`, `NaNException`

Phase 3 — Strategy Pattern and New Solvers

The solver was completely rewritten using the Strategy pattern. New solvers implemented: `NewtonArmijoSolver`, `LevenbergMarquardtSolver`, `BroydenSolver`, `HomotopySolver`. `AbstractSolver` provides shared infrastructure. `SolverConfig` (builder pattern) encapsulates all parameters. `SolverFactory` maps `SolverType` → `SolverStrategy`.

Phase 4 — GSPAR Sparse Solver Port

The 3,259-line Fortran GSPAR code (Grund, WIAS Berlin, ~1994) was ported to Java across four new classes: `SparseMatrix`, `GSPARWorkspace`, `GSPAR`, `GSPARPseudoCode`. The port included: CSR format with DLFASO/DLFATR, structure analysis with pivoting (DLLUGB), pseudo-code generation for LU, forward, and backward passes (DLDPGB/C/D/R/S/T/U/X/Y), and the pseudo-code interpreters (DLDINT, DLSINT). Six bugs from the original Fortran were corrected.

Phase 5 — Bug Fixing and Stabilisation

Over 20 bugs were identified and fixed across the codebase — ranging from critical algorithmic errors (wrong map key in `InitGuessApplier`, race condition in parallel Jacobian) to subtle numerical issues (digit transposition in ISA pressure table, wrong exponent in nozzle area ratio). See the Bug Fix Log for the complete list.

12. Bug Fix Log

Critical Algorithmic Bugs

BF-01: InitGuessApplier — Wrong Map Key

Severity: Critical. The values HashMap uses variable aliases (e.g. `lx_0`) as keys, not variable names. `InitGuessApplier` called `values.replace(varName, value)`. Since `varName` is never a map key, `replace()` silently does nothing. All INIT block values were ignored. The solver ran with default values, not user-specified starting points. **Fix:** `values.replace(varAlias, value)`.

BF-02: evaluateF() Timing — Frozen Residual

Severity: Critical. `fsum` was computed from the old `f` vector (before `applyStep`), so the convergence monitor appeared frozen at the initial residual. **Fix:** `evaluateF()` is called after every `applyStep()`.

BF-03: Jacobian Parallelisation Race Condition

Severity: Critical. `MatrixSimple.setValueAt()` is not thread-safe. Parallel Jacobian construction (threshold `n>50`) caused silent non-deterministic data corruption — sometimes converging, sometimes diverging. **Fix:** `PARALLEL_JACOBI_THRESHOLD = 99999` (disabled).

BF-04: Variable Scaling Applied to values Map

Severity: Critical. `VariableScaler.scaleValues()` modified the same values map that `evaluateF()` reads. Thermodynamic functions received scaled values ($p \approx 1$ Pa instead of 100,000 Pa), immediately producing NaN. **Fix:** `scaleValues()` call removed; feature disabled pending redesign.

Parser Bugs

ID	Location	Description	Fix
BF-06	<code>EqnSysReader</code>	Array detection triggered by unit [m/s] in AS clause	Check only part before " AS " using <code>split(" as ")[0]</code>
BF-07	<code>EqnData.setEquation()</code>	"ATANGES" typo in token split regex (3 places)	Corrected to "ATANGENS"
BF-08	<code>EqnSysReaderDataBlock</code>	<code>getTmpProcDef()</code> returns null → <code>NullPointerException</code>	New <code>initTmpProcDef(name)</code> method that instantiates <code>ProcDef</code> first
BF-09	<code>EqnSysReader</code>	Stray duplicate finally block outside method after inline expansion	Removed stray block

Thermodynamic Function Bugs

ID	Location	Description	Fix
BF-1 0	Bifunctions L656	Nozzle area ratio: Math.pow(fac, -fac) used wrong exponent	Math.pow(fac, -fac1) where $fac1 = (\gamma + 1) / (2(\gamma - 1))$
BF-1 1	Bifunctions L776	ISA pressure: 5474.583 Pa instead of 5474.853 Pa	Digit transposition corrected
BF-1 2	Bifunctions L778	ISA layer index initialised to 1 instead of -1	int index = -1 (signals "not found")

13. Open Issues

13.1 Broyden Quasi-Newton — Resolved via Adaptive Checkpoint/Rollback ✓

Status: Resolved. Broyden with -S:BROYDEN -SB:RESET=10 converges in 11 iterations on the 623-equation benchmark with only 2-3 Jacobian builds instead of 12.

The root cause — $\kappa \approx 10^{14}$ causing sign flip in gradF0 after rank-1 updates — is mitigated by the adaptive checkpoint/rollback strategy: on divergence ($fsum > 2 \times \text{checkpoint } fsum$) the solver restores the pre-update state and halves the interval. After 3 consecutive rollbacks it falls back to pure Newton.

Remaining improvement: GMRES as inner linear solver would allow larger reset intervals without divergence risk. Current Gauss-Jordan is sufficient for the benchmark.

13.2 Variable Scaling — Architecture Incomplete

Scaling flags -SC:DIAGONAL and -SC:UNIT_INTERVAL produce a warning and fall back to -SC:NONE. A correct scaling architecture requires consistent transformation throughout the entire solve loop: scale values → build Jacobian in scaled space → solve → unscale dx → unscale before thermodynamic calls. Step 5 conflicts with physical-unit requirements of thermodynamic functions.

13.3 GMRES Linear Solver — Not Yet Implemented

GMRES is required as the inner linear solver for stable Broyden updates and ILU-preconditioned iterations on near-singular systems. The AbstractSolver architecture is designed to accept a pluggable linear solver — GMRES would slot in as an alternative to Gauss-Jordan and GSPAR in computeStep().

13.4 Block-Structured Hierarchical Variable Names

The parser supports flat variable names only. Infrastructure for dot-notation hierarchy (e.g. turbine.stage1.T_in) is partially in place: unused variables are removed after parsing. Remaining work: LEX-based parser for nested SECTION...END blocks, automatic variable prefixing, and block-triangular decomposition via Tarjan SCC for solver ordering.

13.5 Additional Pending Items

Item	Status	Notes
Jacobian parallelisation	Disabled	MatrixSimple not thread-safe; threshold=99999
Parser line numbers in errors	Pending	ParseException class ready; plumbing needed

Item	Status	Notes
GSPAR first benchmark test	Pending	Compiles and verifies; awaiting runtime test
ILU preconditioner for GMRES	Pending	GSPAR LU as natural preconditioner
SolverState for op. point variation	Pending	Warm-start across -U file runs
// line comments in parser	Pending	Minor parser extension

14. Lessons Learned

14.1 Scaling Cannot Be Added Incrementally

The single most expensive mistake was attempting to add variable scaling as an optional layer on top of an existing architecture. Scaling is a global concern: the values map, the Jacobian, the line search, and the thermodynamic function evaluations must all agree on whether they operate in physical or scaled space. Retrofitting scaling required multiple iterations and ultimately the feature had to be disabled.

Lesson: Design scaling into the architecture from the start, or treat it as a complete system rewrite.

14.2 Adaptive Strategies Outperform Fixed Strategies Under Uncertainty

The initial Broyden implementation diverged with fixed multi-step updates on a 623-equation system with $\kappa \approx 10e14$. Rather than replacing the entire solver, an adaptive checkpoint/rollback strategy resolved the instability: start conservative (interval=1, = Newton), probe aggressively (increase interval on good steps), rollback safely (restore checkpoint on divergence, halve interval). Result: 11 vs 12 iterations, 2-3 vs 12 Jacobian builds on the benchmark.

Lesson: Adaptive strategies that degrade gracefully under failure outperform fixed strategies that assume ideal behaviour. Start conservative, probe aggressively, rollback safely.

14.3 HashMap.replace() Fails Silently on Missing Keys

HashMap.replace(key, value) returns null and does nothing if the key does not exist — no exception, no warning. This caused BF-01 where all initial values were silently discarded. The system appeared to run correctly but used default variable values throughout.

Lesson: Prefer put() over replace() unless "key missing → do nothing" is explicitly desired. Add assertion or LOG.warning when replace() returns null.

14.4 Thread Safety Must Be Verified Before Enabling Parallelism

Enabling parallel Jacobian construction without verifying thread-safety of MatrixSimple.setValueAt() caused silent non-deterministic data corruption (BF-03). The corruption was intermittent — sometimes the solver converged, sometimes diverged — making it extremely hard to diagnose.

Lesson: Never add parallelism to shared mutable state without explicit synchronisation analysis. Non-deterministic bugs are the hardest to find.

14.5 Fortran Port — Sentinel Values and Index Conventions

Fortran uses 1-based array indexing and uses 0 as a sentinel (end of chain). Java uses 0-based indexing. Converting GSPAR required systematically shifting all array accesses by -1 and replacing sentinel 0 with -1. Missing even one conversion causes silent out-of-bounds access in the linked-list traversals.

Lesson: Define a conversion convention before starting a Fortran port and enforce it mechanically throughout — especially for pointer-like linked-list sentinel values.

14.6 Debug Logging Invaluable for Convergence Diagnosis

Adding `LOG.info("DEBUG Iter %d: fsum=%.3e ||dx||=%.3e")` immediately revealed two separate bugs: the frozen fsum (BF-02, evaluateF timing) and the Broyden sign flip. Without this, both would have been dismissed as "solver does not converge" without actionable diagnosis.

Lesson: Instrument convergence loops with detailed per-iteration logging from the beginning, not as an afterthought. The cost is negligible; the diagnostic value is enormous.

14.7 The Equation-Oriented Approach Scales — Sequential-Modular Does Not

The SpeedUp/Aspen approach of solving all equations simultaneously is the right architecture for complex coupled aerothermodynamic systems. Sequential-modular simulation (solve component by component) breaks down for systems with strong recycle streams and feedback loops. CMDSolver's equation-oriented design handles these naturally without special recycle handling logic.

Lesson: For tightly coupled systems, invest in the equation-oriented infrastructure upfront — it pays back immediately in generality and robustness.

14.8 Symbolic Differentiation Accurate but Needs Analytical Complement

CASprzak provides exact symbolic derivatives, eliminating finite-difference errors. However, it cannot differentiate built-in procedure calls (`cpOfT`, `htofTTref`, etc.). These require analytical derivative implementations in `Bifunctions`. Finite-difference fallback (adaptive $h = \max(10e-4 \cdot |x|, 10e-6)$) was implemented for the 19 locations where symbolic derivatives are unavailable.

Lesson: CAS-based differentiation is excellent for algebraic equations. Supplement with carefully implemented analytical derivatives for complex built-in functions.

15. Roadmap

Priority	Item	Status	Notes
P1	Test GSPAR on 623-eq benchmark	Pending	First real-world validation of Java GSPAR port
P1	GMRES linear solver	Pending	Required for stable Broyden; architecture ready
P2	Broyden + GMRES integration	Pending	Replaces Gauss-Jordan in BroydenSolver
P2	Variable scaling (redesign)	Pending	Transformer pattern around evaluateF()
P2	Parser line numbers in errors	Pending	ParseException infrastructure ready
P3	LEX hierarchical parser	Pending	Enables dot-notation, block structure, Tarjan SCC
P3	ILU preconditioner for GMRES	Pending	GSPAR LU structure as natural preconditioner
P3	Thread-safe Jacobian parallelism	Pending	Per-row local arrays; speedup for n>500
P4	SolverState for op. point variation	Pending	Warm-start across -U runs
P4	// line comments in parser	Pending	Minor parser extension